```
;=================================================================
; CIS11 - Bubble Sort Sorcerers
; Team: Bryant Martinez Fossati, Cassandra Suarez, Ryland Tolentino
; Class: CIS11
;
; LC-3 Bubble Sort  Complete Implementation
;
; Input:  User types 8 integers (0-100) at keyboard, Enter after each
; Output: 8 sorted integers displayed in ascending order on console
;
; Test input : 11  8  2  17  6  4  3  21
; Expected   :  2  3  4   6  8 11 17  21
;
; ---- Register usage (main program) ----
;   R0 = scratch / TRAP I/O parameter
;   R1 = array pointer
;   R2 = loop counter (outer sort / input / output)
;   R3 = inner loop counter (sort)
;   R4 = A[j]   (current element during sort)
;   R5 = A[j+1] (next element during sort)
;   R6 = stack pointer  (dedicated; never reused for other purpose)
;   R7 = return address (set by JSR, restored by RET)
;
; ---- Subroutines ----
;   INPUT_NUM   reads one decimal integer from keyboard (ASCII->binary)
;   PRINT_NUM   prints one decimal integer to console  (binary->ASCII)
;
; ---- Memory layout ----
;   x3000+ : program code + constants
;   ARRAY   : 8-word array (input values / sorted result)
;   STACK   : 20-word software stack (grows downward)
;   STTOP   : stack pointer initialised here (empty-stack sentinel)
;=================================================================

        .ORIG x3000

;-----------------------------------------------------------------
; INITIALIZATION  load stack pointer
;-----------------------------------------------------------------
MAIN    LEA  R6, STTOP          ; R6 = stack pointer (points above stack)

;-----------------------------------------------------------------
; INPUT PHASE  read 8 integers from keyboard into ARRAY
;-----------------------------------------------------------------
        LEA  R1, ARRAY          ; R1 = pointer to start of array
        AND  R2, R2, #0
        ADD  R2, R2, #8         ; R2 = input count = 8

IN_MAIN BRnz SORT_START         ; if count <= 0, done reading
        JSR  INPUT_NUM          ; call subroutine  result returned in R0
```

```
        STR  R0, R1, #0         ; store entered value into array[i]
        ADD  R1, R1, #1         ; advance array pointer to next slot
        ADD  R2, R2, #-1        ; decrement remaining count
        BRnzp IN_MAIN           ; loop back for next number

;---------------------------------------------------------------
; SORT PHASE  in-place bubble sort, n=8 fixed, 7 passes
;---------------------------------------------------------------
SORT_START
        AND  R2, R2, #0
        ADD  R2, R2, #7         ; R2 = outer counter = n-1 = 7

OUTER   BRnz PRINT_START        ; if outer counter <= 0, sorting done
        LEA  R1, ARRAY          ; reset array pointer to start each pass
        ADD  R3, R2, #0         ; R3 = inner counter for this pass

INNER   BRnz NXTPASS            ; if inner counter <= 0, this pass done
        LDR  R4, R1, #0         ; R4 = A[j]    load current element
        LDR  R5, R1, #1         ; R5 = A[j+1]  load next element
        ; Compute A[j] - A[j+1] via 2's complement subtraction
        NOT  R0, R5             ; R0 = bitwise NOT of A[j+1]
        ADD  R0, R0, #1         ; R0 = -A[j+1]
        ADD  R0, R4, R0         ; R0 = A[j] - A[j+1]  (sets condition codes)
        BRnz NOSWAP             ; if A[j] <= A[j+1], no swap needed
        ; Swap A[j] and A[j+1]
        STR  R5, R1, #0         ; M[j]   = old A[j+1]
        STR  R4, R1, #1         ; M[j+1] = old A[j]
NOSWAP  ADD  R1, R1, #1         ; advance pointer to next pair
        ADD  R3, R3, #-1        ; decrement inner counter
        BRnzp INNER             ; next comparison

NXTPASS ADD  R2, R2, #-1        ; decrement outer counter
        BRnzp OUTER             ; next pass

;---------------------------------------------------------------
; OUTPUT PHASE  print 8 sorted values to console
;---------------------------------------------------------------
PRINT_START
        LEA  R1, ARRAY          ; R1 = pointer to sorted array
        AND  R2, R2, #0
        ADD  R2, R2, #8         ; R2 = print count = 8

PR_MAIN BRnz DONE               ; if count <= 0, done printing
        LDR  R0, R1, #0         ; R0 = next sorted value
        JSR  PRINT_NUM          ; call subroutine to print it
        AND  R0, R0, #0
        ADD  R0, R0, #10        ; R0 = x000A = newline
        TRAP x21                ; OUT: newline after each number
        ADD  R1, R1, #1         ; advance array pointer
        ADD  R2, R2, #-1        ; decrement count
```

```
        BRnzp PR_MAIN           ; loop

DONE    TRAP x25                ; HALT


;===============================================================
; SUBROUTINE: INPUT_NUM
;
; Reads one decimal integer from the keyboard (range 0 to 100).
; Uses TRAP x23 (GETC) to read one character at a time.
; Accepts digit characters '0'-'9'; echoes each digit via TRAP x21.
; Any non-digit (Enter, CR, space) terminates input.
; Multi-digit assembly: total = total * 10 + digit
;   Multiply by 10 uses shift-add:  total*8 + total*2
;
; Input  : none
; Returns: R0 = binary integer value entered by user
; Saves  : R1, R2, R3, R7  (pushed on stack; popped before return)
;===============================================================
INPUT_NUM
        ; --- PUSH: save registers this subroutine will modify ---
        ADD  R6, R6, #-1
        STR  R7, R6, #0         ; push return address (TRAP x23 overwrites R7)
        ADD  R6, R6, #-1
        STR  R1, R6, #0         ; push R1
        ADD  R6, R6, #-1
        STR  R2, R6, #0         ; push R2
        ADD  R6, R6, #-1
        STR  R3, R6, #0         ; push R3

        AND  R1, R1, #0         ; R1 = running total = 0

DGTLOOP TRAP x23                ; GETC: read one character into R0
        ; ---- Check lower bound: char >= '0' (x30 = 48) ----
        LD   R3, NEG_ZERO       ; R3 = -48
        ADD  R2, R0, R3         ; R2 = R0 - '0'
        BRn  IN_DONE            ; if R2 < 0: character is below '0'  stop
        ; ---- Check upper bound: char <= '9' (x39 = 57) ----
        ADD  R3, R2, #-9        ; R3 = (R0 - '0') - 9
        BRp  IN_DONE            ; if R3 > 0: character is above '9'  stop
        ; ---- Valid digit: echo it so user sees what they typed ----
        TRAP x21                ; OUT: echo the digit character (R0 unchanged)
        ; ---- Accumulate: R1 = R1 * 10 + digit ----
        ;      R2 holds the digit value (0-9)
        ;      R1 * 10  =  (R1 * 8) + (R1 * 2)
        ADD  R3, R1, R1         ; R3 = R1 * 2
        ADD  R3, R3, R3         ; R3 = R1 * 4
        ADD  R3, R3, R3         ; R3 = R1 * 8
        ADD  R3, R3, R1         ; R3 = R1 * 9
        ADD  R3, R3, R1         ; R3 = R1 * 10
```

```
        ADD  R1, R3, R2         ; R1 = R1*10 + digit
        BRnzp DGTLOOP           ; read next digit

IN_DONE ADD  R0, R1, #0         ; move result from accumulator R1 into R0

        ; --- POP: restore registers in reverse push order ---
        LDR  R3, R6, #0
        ADD  R6, R6, #1         ; pop R3
        LDR  R2, R6, #0
        ADD  R6, R6, #1         ; pop R2
        LDR  R1, R6, #0
        ADD  R6, R6, #1         ; pop R1
        LDR  R7, R6, #0
        ADD  R6, R6, #1         ; pop return address
        RET                     ; return to caller


;===============================================================
; SUBROUTINE: PRINT_NUM
;
; Prints one integer (range 0 to 100) as decimal ASCII digits.
; Performs binary-to-ASCII conversion using repeated subtraction:
;   Value 100      ? special case, prints "100"
;   Value 10-99    ? extract tens via repeated subtraction of 10,
;                    remainder is ones digit
;   Value  0-9     ? print single digit
; Each digit sent to console with TRAP x21 (OUT) after adding x30.
;
; Input  : R0 = integer to print (0 to 100)
; Returns: nothing (side effect: characters printed to console)
; Saves  : R0, R1, R2, R3, R7  (pushed on stack; popped before return)
;===============================================================
PRINT_NUM
        ; --- PUSH: save registers this subroutine will modify ---
        ADD  R6, R6, #-1
        STR  R7, R6, #0         ; push return address
        ADD  R6, R6, #-1
        STR  R0, R6, #0         ; push R0
        ADD  R6, R6, #-1
        STR  R1, R6, #0         ; push R1
        ADD  R6, R6, #-1
        STR  R2, R6, #0         ; push R2
        ADD  R6, R6, #-1
        STR  R3, R6, #0         ; push R3

        ADD  R1, R0, #0         ; R1 = working copy of value to print

        ; ---- Special case: value == 100 ----
        LD   R2, NEG_100
        ADD  R2, R1, R2         ; R2 = value - 100
```

```
        BRnp CHK_TENS           ; if result != 0, value is not 100
        ; value IS 100   print '1', '0', '0'
        LD   R0, ASCII_ONE
        TRAP x21                ; print '1'
        LD   R0, ASCII_ZERO
        TRAP x21                ; print '0'
        TRAP x21                ; print '0'
        BRnzp PR_DONE

        ; ---- Two-digit check: value >= 10 ? ----
CHK_TENS
        LD   R3, NEG_TEN
        ADD  R3, R1, R3         ; R3 = value - 10
        BRn  ONE_DIGIT          ; if value < 10, go to single-digit path

        ; ---- Two-digit path: extract tens and ones ----
        AND  R2, R2, #0         ; R2 = tens digit = 0
TENSLOOP
        LD   R3, NEG_TEN
        ADD  R3, R1, R3         ; R3 = R1 - 10
        BRn  TENS_DONE          ; if R1 < 10, done extracting tens
        ADD  R1, R3, #0         ; R1 = R1 - 10   (subtract one ten)
        ADD  R2, R2, #1         ; tens digit++
        BRnzp TENSLOOP

TENS_DONE
        ; R2 = tens digit (1-9),  R1 = ones digit (0-9)
        LD   R0, ASCII_ZERO
        ADD  R0, R0, R2         ; R0 = '0' + tens  (binary-to-ASCII)
        TRAP x21                ; OUT: print tens digit
        LD   R0, ASCII_ZERO
        ADD  R0, R0, R1         ; R0 = '0' + ones  (binary-to-ASCII)
        TRAP x21                ; OUT: print ones digit
        BRnzp PR_DONE

        ; ---- Single-digit path: value 0-9 ----
ONE_DIGIT
        LD   R0, ASCII_ZERO
        ADD  R0, R0, R1         ; R0 = '0' + value  (binary-to-ASCII)
        TRAP x21                ; OUT: print the digit

PR_DONE
        ; --- POP: restore registers in reverse push order ---
        LDR  R3, R6, #0
        ADD  R6, R6, #1         ; pop R3
        LDR  R2, R6, #0
        ADD  R6, R6, #1         ; pop R2
        LDR  R1, R6, #0
        ADD  R6, R6, #1         ; pop R1
        LDR  R0, R6, #0
```

```
        ADD  R6, R6, #1         ; pop R0
        LDR  R7, R6, #0
        ADD  R6, R6, #1         ; pop return address
        RET                     ; return to caller


;---------------------------------------------------------------
; CONSTANTS  used by LD instruction (PC-relative addressing)
;---------------------------------------------------------------
NEG_ZERO   .FILL xFFD0          ; -48  (negation of ASCII '0' = x0030)
NEG_TEN    .FILL xFFF6          ; -10
NEG_100    .FILL xFF9C          ; -100
ASCII_ZERO .FILL x0030          ; '0'  (ASCII base for digit conversion)
ASCII_ONE  .FILL x0031          ; '1'  (used for printing 100)

;---------------------------------------------------------------
; DATA
;---------------------------------------------------------------
ARRAY   .BLKW 8                 ; 8-word array for input values / sorted result

;---------------------------------------------------------------
; STACK  software stack, grows downward
; Each PUSH: ADD R6,R6,#-1  then  STR Rx,R6,#0
; Each POP:  LDR Rx,R6,#0   then  ADD R6,R6,#1
; Max stack depth = 20 words (INPUT_NUM uses 4, PRINT_NUM uses 5)
;---------------------------------------------------------------
        .BLKW 20                ; 20-word stack buffer
STTOP   .FILL x0000             ; stack pointer initialised to this address

        .END
```